

track this file, we use `git add <file>` in the working directory. This moves `file_1.txt` to the staging area, as shown in Figure 8.

To commit the changes staged, `git commit -m "<commit-message>"` is used, as shown in Figure 9.

Viewing changes

`git status` is used to see the files that are in the working directory and in the staging area. The files under the title 'Changes to be committed' show the staged files and the ones under 'Untracked files' show the files in the working directory, as in Figure 10.

To view the changes made to the tracked files in the working directory and not yet staged, we use `git diff`. It shows the lines added and removed from the previous snapshot, as shown in Figure 11.

And after doing `git add <file>`, to view the changes made to the tracked files present in the staging area that are to be committed, we do `git diff --cached`, which is shown in Figure 12. This is followed by a `git commit -m "Add file2. 2nd commit."`

Viewing history

After doing several commits or cloning an existing repository, in order to see the commit history, `git log` is used, which is shown in Figure 13.

There are several flags that can be used. For example, `git log --pretty --oneline` to see the history with short one-liner commit messages, `git log --stat` to see stats associated with each commit, and `git log -p` to view patches introduced per commit.

Undoing changes

One of the great things about a VCS along with tracking the changes is being able to undo them whenever necessary. If you forget to add a file in a commit, it can be included in the commit using `git commit --amend`. This will open the default text editor associated with Git with the commit message mentioned in the commit, previously. You can change the commit

```
(base) ekta@git-demo$ echo "Hello! I am content in file_1" > file_1.txt
(base) ekta@git-demo$ git add file_1.txt
```

Figure 8: Demonstration of `Git add`

```
(base) ekta@git-demo$ git commit -m "First commit"
[master (root-commit) 204bb8e] First commit
1 file changed, 1 insertion(+)
create mode 100644 file_1.txt
```

Figure 9: Demonstration of a Git commit

```
(base) ekta@git-demo$ echo "Working dir file" > file_2.txt
(base) ekta@git-demo$ echo "line number 2 in file_1" >> file_1.txt
(base) ekta@git-demo$ git add file_1.txt
(base) ekta@git-demo$ git status
On branch master
Changes to be committed:
  (use "git reset HEAD <file>..." to unstage)

        modified:   file_1.txt

Untracked files:
  (use "git add <file>..." to include in what will be committed)

        file_2.txt
```

Figure 10: Demonstration of `git status`: removed lines (red), added lines (green)

```
(base) ekta@git-demo$ echo "lets demo diff command" >> file_1.txt
(base) ekta@git-demo$ git diff
diff --git a/file_1.txt b/file_1.txt
index 4551ca9..58a2c72 100644
--- a/file_1.txt
+++ b/file_1.txt
@@ -1,2 +1,3 @@
-Hello! I am content in file_1
+line number 2 in file_1
+lets demo diff command
```

Figure 11: Demonstration of `Git diff`

```
(base) ekta@git-demo$ git add file_1.txt file_2.txt
(base) ekta@git-demo$ git diff --cached
diff --git a/file_1.txt b/file_1.txt
index ed95119..4f65ab1 100644
--- a/file_1.txt
+++ b/file_1.txt
@@ -1 +1,2 @@
-Hello! I am content in file_1
+line number 2 in file_1
+lets demo diff command
diff --git a/file_2.txt b/file_2.txt
new file mode 100644
index 0000000..bde607e
--- /dev/null
+++ b/file_2.txt
@@ -0,0 +1 @@
+Working dir file
```

Figure 12: Demonstration of `Git diff --cached`

```
(base) ekta@git-demo$ git log
commit 3fd9bd336be87862cf2e8acdb53147106ea0533b (HEAD -> master)
Author: Ekta Nandwani <Ekta.Nandwani@iiitb.org>
Date: Thu May 7 19:21:07 2020 +0530

    Add file2. 2nd commit

commit 204bb8ec035435c19eb4c51a090da406e8e7d921
Author: Ekta Nandwani <Ekta.Nandwani@iiitb.org>
Date: Thu May 7 19:10:46 2020 +0530

    First commit
```

Figure 13: Commit history